Searching for TCM codes using genetic algorithms

I. Soto
R.A. Carrasco

Indexing terms: Code design, Searching, Genetic algorithm

Abstract: The authors aim to show a new approach to the search for convolutional codes using genetic algorithms. By the use of genetic algorithms, automatic classification of the convolutional codes and less computationally demanding algorithms are required to find and evaluate the most suitable codes. Also, code performance and complexity are evaluated.

1 Introduction

Usually, the search algorithms used to find optimal codes in trellis coded modulation (TCM) are based on an exhaustive search over the code space with high computational demands [1, 2, 4, 5]. An alternative approach using a genetic algorithm is presented to search for TCM codes.

A class can be defined as a particular code property over the search space; and classification will be defined as the process that assigns a class to one code. Traditionally in TCM, an exhaustive search algorithm is used which classifies the codes over a geometrically uniform partition using set partition mapping [9], according to the degree of symmetry of the code [5, 13, 14]. There is an alternative approach using convolutional codes over the ring of integers modulo-q, which does not use the set partitioning rule, and the categories are restricted to one which is called transparent [3, 4, 12].

In both the traditional and alternative approaches, a better performance of the search algorithms is achieved by the introduction of the classes described, which reduces the complexity of the search, since smaller search spaces are considered.

2 TCM model

This Section describes a multilevel convolutional encoder (MCE), defined over a general space S, which can be any nondense set, such as the binary set or the ring set. The MCE is characterised by the generator matrix $G(D)$, which maps the input information frames onto the output coded frames x_n, and can be written in the functional form of eqn. 1 [3, 4]:

$$G(D) = \begin{bmatrix} g^1(D)/f(D) \\ I_k \ldots \\ g^k(D)/f(D) \end{bmatrix} \tag{1}$$

The entries of the matrix $G(D)$ are the feedforward polynomial $g^{(i)}(D) = g_s^{(i)} D^s + \ldots + g_1^{(i)} D + g_0^{(i)}$ and the feedback polynomial $f(D) = f_s D^s + \ldots + f_1 D + f_0$, where the coefficients $g_s^{(i)}, f_s \in S$, D is the delay operator and I_k is the $(m \times m)$ identity matrix.

For example, in the case of the ring-TCM multilevel convolutional encoder with rate (p/m), which is defined over the searching space $S = \mathbf{Z}_M$ ring of integers modulo-M. The MCE is a time-invariant linear finite-state sequential circuit that, having $m \in \mathbf{Z}_M$ information input symbols $(a_1, a_2, \ldots, a_m)$ at a time, generates $p \in \mathbf{Z}_M$ encoded output symbols $(x_1, x_2, \ldots, x_p)$ at a time, where the coefficients of the MCE also belong to $\mathbf{Z}_M$, and all arithmetic operations satisfy the properties of the ring of integers modulo-M. To achieve the same transmission rate as uncoded modulation, the value of p is chosen to be $m + 1$. The modulo-M ring-TCM MCE suitable for M-PSK (phase shift keying) modulation is shown in Fig. 1.

The ring-TCM MCE suitable for the rectangular M-QAM (quadrature amplitude modulation) signal set is rather similar to the ring-TCM MCE for M-PSK modulation, with the following differences. First, the code-to-signal mapping must properly assign the output of the MCE to the two signals of the 2-D M-QAM constellation. The two-coded 2-D M-QAM signals are then modulated and sent to the channel using time division, forming the 4-D coded signal. Secondly, the 4-D ring-TCM codes suitable for the rectangular M-QAM constellation are always defined over the ring of integers modulo-4. The modulo-M ring-TCM MCE suitable for M-QAM modulation is shown in Fig. 2.

The asymptotic coding gain over a Gaussian channel is described as follows. Let M_c and M_u be the number of signal symbol elements of the coded and uncoded schemes respectively; R_c the coding rate, d_{free} and d_u the minimum squared Euclidean distance of the coded and uncoded schemes, respectively. Then, the asymptotic coding gain is given by [4]:

$$g_\infty = 10 \log_{10} \left[\frac{\log M_c}{\log M_u} R_c \frac{d_{free}}{d_u} \right] \tag{2}$$

where g_∞ is used to evaluate the performance of the coded modulation schemes over the reference uncoded modulation scheme.

© IEE, 1997

IEE Proceeedings online no. 19971039

Paper first received 8th November 1995 and in revised form 15th November 1996

The authors are with the Electronic Division, School of Engineering, Staffordshire University, PO Box 333, Beaconside, Stafford ST18 0AD, UK

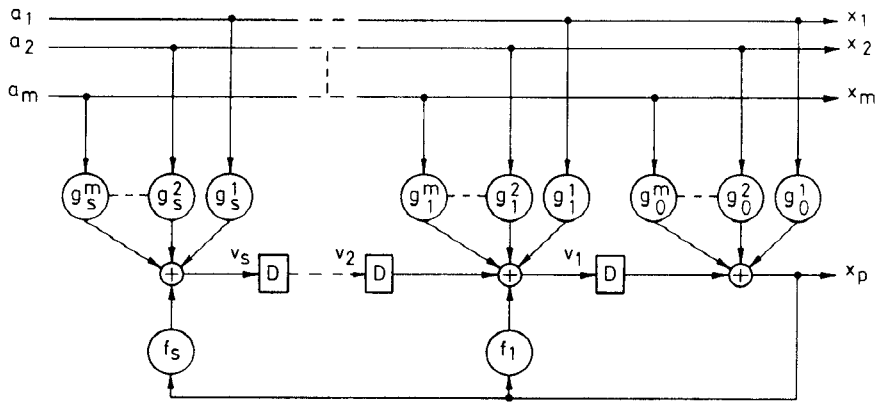

Fig. 1 *Ring-TCM MCE for M-PSK modulation*

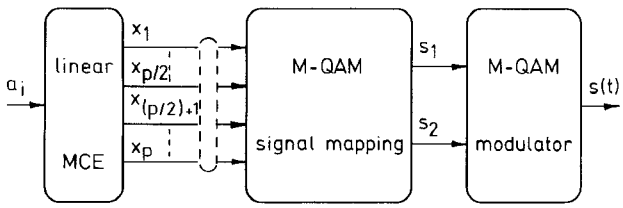

Fig. 2 *Block diagram of 4-D ring TCM for M-QAM transmitter*

3 Search problem

The general formulation for search TCM codes over a Gaussian channel can be described by:

$$\max d_{free}\{g_s^{(i)}, \ldots, g_0^{(i)}, f_s, \ldots, f_1\} \tag{3}$$

$$\min n_{free}\{g_s^{(i)}, \ldots, g_0^{(i)}, f_s, \ldots, f_1\} \tag{4}$$

subject:

$$g_s^{(i)}, \ldots, g_0^{(i)} \in S$$

$$f_s, \ldots, f_1 \in S$$

where n_{free} is the average number of nearest neighbours [5].

4 Genetic algorithm

For the solution of optimisation problems, a genetic algorithm has been investigated for several applicatiuons in signal processing such as speech compression, and shown to be effective at exploring a large and complex search space in an adaptive way [8]. Next, a formal definition of genetic algorithms and the operators used in this work are described.

4.1 Def: genetic algorithm

A genetic algorithm can be defined as the nine-tuple:

GA	= $(p^0, S, \lambda, L, f, s, c, m, T)$ where
$p^0 = (a_1^0, \ldots, a_\lambda^0)$	= initial population for $t = 0$; in the tth generation $p^t = (a_1^t, \ldots, a_\lambda^t)$
S	= search space in which the chromosome is encoded
$a_\lambda^0 \in S$	= elements of the search space
$\lambda \in N$	= population size
$L \in N$	= length of chromosome
f: $S \rightarrow R$	= fitness function
s: $S^\lambda \rightarrow S$	= selection operator
c: $S \times S \rightarrow S \times S$	= crossover operator
m: $S \rightarrow S$	= mutation operator
T: $S^\lambda \rightarrow \{0, 1\}$	= termination criterion

In the TCM search code, the genetic algorithm basically selects and classifies the generator codeword with a high degree of adaptation as parents generate a new generation by the combination of components from the parents, and by the elimination of the weakest generators from the population. In genetic algorithms there are many operators [7, 11], but the selection, crossover and mutation operators have been chosen.

4.2 Selection operator

Assuming that the initial population $p^0 = (a_1^0, \ldots, a_\lambda^0)$ can be obtained in a heuristic or random way, from the search space S in which the chromosome is encoded, the next generation is obtained from members of the previous generation using a stochastic process, which guarantees that the number of times that one structure is selected is dependent on its performance compared with the rest of the population.

The parent selection operation s, s: $S^\lambda \rightarrow S$, produces an intermediate population $p'^t = (a_1'^t, \ldots, a_\lambda'^t)$ from the population $p^t = (a_1^t, \ldots, a_\lambda^t)$ in the tth generation, $s(p^t) = p'^t$. Any $a_i'^t = a_q^t$ in the p'^t is selected by a given random number α_i satisfying the following condition:

$$0 \leq \alpha_i \leq \sum_{j=1}^{\lambda} f(a_j^t) \tag{5}$$

The index q is obtained from:

$$q = \max\left\{k/\forall k \in \{1, \ldots, \lambda\}, s.t. \alpha_i \leq \sum_{k=1}^{\lambda} f(a_k^t)\right\} \tag{6}$$

where $\Sigma_{k=1}^{\lambda} f(a_k^t)$ is the summation of all the fitness from population p'^t for all members of the population picking up the first index k that reaches $\alpha_i \leq \Sigma_{k=1}^{\lambda} f(a_k^t)$, the Algorithm 1 for selection operator is shown in the Appendix.

4.3 Crossover operator

For any selected chromosomes in a population p^t, an associated real value, $0 \leq \rho \leq 1$, is generated randomly. If ρ is greater than the defined crossover threshold ρ_c where $0 \leq \rho_c \leq 1$, the crossover operator c: $S \times S \rightarrow S \times S$ is applied to this pair of chromosomes [10].

The strategy used in this work is the one-point crossover c_{op}, which produces an intermediate population p'^t from the population p^t and is defined below:

$$\begin{pmatrix} a_i'^t \\ a_{(i+1)}'^t \end{pmatrix} = C_{op}\left(\begin{pmatrix} a_i^t \\ a_{(i+1)}^t \end{pmatrix}\right) \forall a_i^t \in p^t/i \in \{1, \ldots, \lambda\}$$

$$= C_{op}\left(\begin{pmatrix} [a_{i,\rho_1}, a_{i,\rho_2}, \ldots, a_{i,\rho_j}, a_{i,(\rho_{j+1})}, \ldots, a_{i,\rho_L}]^T \\ [a_{(i+1),\rho_1}, a_{(i+1),\rho_2}, \ldots, a_{(i+1),\rho_j}, a_{(i+1),(\rho_{j+1})}, \ldots, a_{(i+1),\rho_L}]^T \end{pmatrix}\right)$$

$$= C_{op}\begin{pmatrix} [a_{i,\rho_1}, a_{i,\rho_2}, \ldots, a_{i,\rho_j}, a_{(i+1),(\rho_{j+1})}, \ldots, a_{(i+1),\rho_L}]^T \\ [a_{(i+1),\rho_1}, a_{(i+1),\rho_2}, \ldots, a_{(i+1),\rho_j}, a_{i,(\rho_{j+1})}, \ldots, a_{i,\rho_L}]^T \end{pmatrix} \tag{7}$$

The pair of chromosomes are separated into two subchromosomes at ρ_j, whence a new pair are composed by swapping the second subchromosome, where each one is crossed from ρ_{j+1} to ρ_L. The Algorithm 2 for crossover operator is shown in the Appendix.

4.4 Mutation operator

For any chromosome in a population p^t, an associated real value $0 \le \rho \le 1$ is generated randomly. If ρ is less than the defined mutation threshold ρ_m, where $0 \le \rho_m \le 1$, the mutation operator is applied to the chromosome. The mutation operator simply alters one bit from 0 to 1 (or 1 to 0) in a chromosome. The mutation operator m: $S \rightarrow S$ produces an intermediate population p'^t from the population p^t, as below:

$$a_i'^t = m(a_i^t) \quad \forall i \in \{1, \ldots, \lambda\}$$

$$a_{i,\rho_k}'^t = \begin{cases} a_{i,\rho_k} & \text{for } k \in \{1, 2, \ldots, p-1, p+1, \ldots, L\} \\ \bar{a}_{i,\rho_k} & \text{for } k = p \end{cases} \tag{8}$$

where $1 \le p \le L$. Algorithm 3 for the mutation operator is shown in the Appendix.

The reproduction and crossover operators give to the genetic algorithms the bulk of their processing power. The mutation operator is needed because, even though reproduction and crossover are the main operators, occasionally they may become overzealous and lose some potentially useful genetic material.

Usually there are four parameters to control the evolution of the genetic algorithm. They are: the population size $\lambda \in N$, the crossover threshold ρ_c, mutation threshold ρ_m and the number of generations. If three of them are kept fixed, an optimum value for the free parameter can be found in order to produce the optimum code rate, d_{free} and n_{free} distance.

The following is an example to show the way in which the operators of the genetic algorithms are used to search for TCM codes. After this example, a comparison of the exhaustive search algorithm and the genetic algorithm is made.

4.5 Example

Let $V = \{0, 1\}$ be the binary source alphabet for ordinary TCM and A and B two chromosomes. Then $I = \{0, 1\}^L$ is the closure of alphabet V with L the length of chromosome defined as before, and $A = a_1a_2a_3a_4a_5a_6$ and $B = b_1b_2b_3b_4b_5b_6$ where a_i and b_i are genes, with a_i, $b_i \in V$.

Let us define two pointers cp the crosspoint pointer and mp the mutation pointer. The values that mp can take are $J = \{1, 2, 3, 4, 5, 6\}$, the index set of one chromosome. The values that cp can take are $J \cup \{0\}$.

Assuming that A and B have been selected as parents of two children C and D, and that cp = 3, then after the crossover operator is applied to A and B, $C = a_1a_2a_3b_4b_5b_6$ and $D = b_1b_2b_3a_4a_5a_6$. The reproduction operator is a particular case of the crossover operator, if cp = 0, then $C = A$ and $D = B$, or $C = B$ and $D = A$.

Now assuming mp = 4, then the mutation of the chromosome $A = a_1a_2a_3a_4a_5a_6$ is $E = a_1a_2a_3\hat{a}_4a_5a_6$, where E is a chromosome and $\hat{a}_4 = 0$ if $a_4 = 1$ or $\hat{a}_4 = 1$ if $a_4 = 0$.

The exhaustive search algorithm for the TCM model has been considered elsewhere [1, 2, 4, 6, 12]. The benefit is that the multilevel convolutional codes designed by this method achieve better asymptotic coding gains. For example, the code $g_0^1 = 2$, $g_1^1 = 1$, $f_1 = 2$, denoted by (21/2), has an asymptotic coding gain of 3.01 dB over uncoded 2-PSK.

In general, and in accordance with Fig. 1, the codes are denoted by $(g_0^1 \ldots g_0^m\, g_1^1 \ldots g_1^m \ldots g_s^1 \ldots g_s^m / f_1 \ldots f_s)$ in this paper.

5 Discussion of results

The disadvantage of the exhaustive search algorithm is the time taken to generate codes, especially when a high number of states and a high order constellation are used. It is important to note that the exhaustive search for the rectangular M-QAM constellation is more computationally intensive than that for M-PSK constellations, because the rectangular M-QAM constellation requires that, to determine d_{free} and n_{free}, all paths in the trellis be examined and compared to each other.

The fitness function used in this work is *a priori* function, based on the computation of the distances. Once the code is obtained, a further check of the code is required to investigate the performance in terms of symbol error rate as a function of signal to noise ratio.

The application of the genetic algorithm to 16-QAM is shown with new codes for this particular modulation scheme to demonstrate its feasibility, when the number of states is increased to produce better constraint lengths. The application of the genetic algorithm to the code-to-signal mapping for 16-QAM has produced a range of 180° and 90° rotational invariant ring-TCM (RI ring-TCM), which are presented in Table 1, and compared with the exhaustive algorithm.

Table 1: RI ring-TCM codes for 16-QAM

n_{st}	Ring-TCM code	Rot	d_{free}	n_{free}	g_∞ dB
2	(030 020 / 0)	90°	4.0	19.187	3.01
4	(220 030 / 2)	90°	4.0	5.937	3.01
◆4	(020 300 / 0)	90°	4.0	6.5	3.01
*8	(020 212 010 / 32)	90°	4.0	1.031	3.01
*8	(022 231 013 / 03)	90°	4.0	0.75	3.01
*8	(010 232 020 / 21)	180°	5.0	5.234	3.98
◆8	(000 311 311 / 03)	180°	4.0	4.5	3.98
*16	(010 232 010 / 22)	90°	6.0	9.796	4.77
◆16	(233 311 213 / 20)	90°	10.0	0.5	4.77

In this Table, d_{free} is the minimum squared Euclidean distance between coded sequences of the ring-TCM code suitable for 16-QAM, g_∞ is the asymptotic coding gain of the ring-TCM code suitable for 16-QAM over uncoded 8-AMPM modulation, (*) means that the exhaustive search was stopped and (◆) means a new generated code using the genetic algorithm.

The number of codes that can be found for ring-TCM 4-PSK with different values of d_{free} are shown in Fig. 3. This Figure has been produced after running the exhaustive algorithm for a long period of time and will be used as a reference in the comparison of the exhaustive search algorithm and the genetic algorithm.

According to Fig. 3, there are many codes that can be found for different distances, and further checking of each code is needed according to other parameters such as n_{free} and g_∞ in the Gaussian channel or other parameters when searching for fading channels. By defining the repetition factor as the number of times that the same code is found by a search algorithm,

Fig. 4 shows the number of times that each code is found when carrying out an exhaustive search. The worst case is a repetition factor of 200.

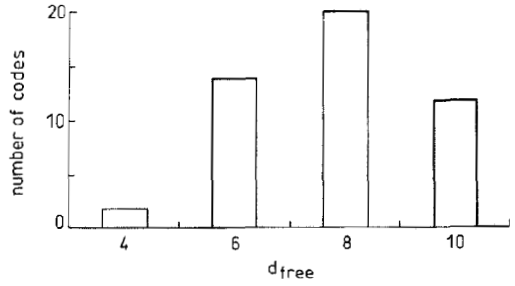

Fig. 3 *Number of codes for ring-TCM 4-PSK*

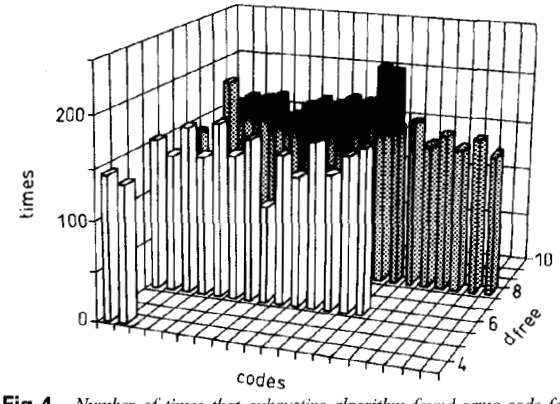

Fig. 4 *Number of times that exhaustive algorithm found same code for ring-TCM 4-PSK*

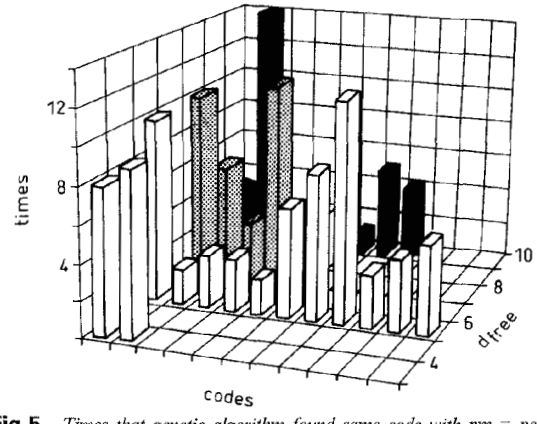

Fig. 5 *Times that genetic algorithm found same code with pm = pc = 0.5, and 40 codewords, for ring-TCM 4-PSK*

Fig. 5 shows the number of times that the same code was found using the genetic algorithm. This bar chart corresponds to the case when $pm = pc = 0.5$, and the population size was 40 codewords. With the genetic algorithm, computer processing is reduced when the repetition of the code is reduced and the search in the space is reduced.

Fig. 6 shows the nondeterministic nature of the genetic algorithm. In this particular experiment the genetic algorithm could not find all the 12 codes for $d_{free} = 10$ shown in Fig. 3. But using hundreds of times fewer repetitions, this algorithm can achieve the same results when it is run two or three times, each with a different population size.

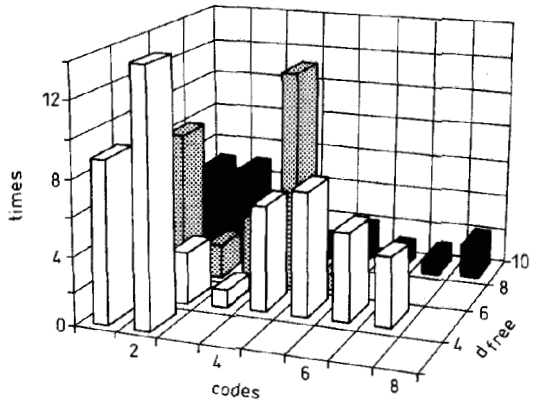

Fig. 6 *Times that genetic algorithm found same code with pm = pc = 0.5, and 20 codewords, for ring-TCM 4-PSK*

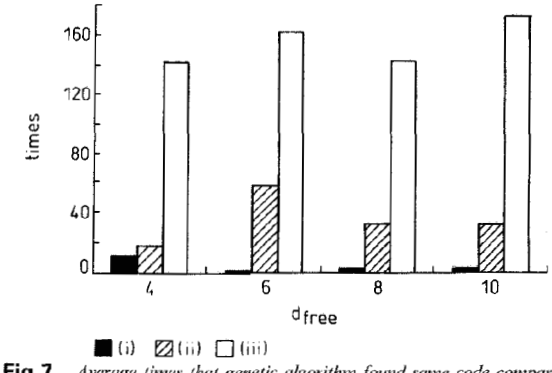

Fig. 7 *Average times that genetic algorithm found same code compared with the exhaustive algorithm, for ring-TCM 4-PSK*
(i) population = 20
(ii) population = 40
(iii) exhaustive

Fig. 7 shows an improvement of approximately 170, for $d_{free} = 10$, in the performance of genetic algorithm compared with the exhaustive algorithm, for populations of 20 and 40 codes.

6 Conclusions

This paper has presented an alternative approach to the search for convolutional codes based on genetic algorithms. The codes found by genetic algorithms compared with codes generated by the exhaustive search algorithm show an improvement of approximately 170 in the case of ring-TCM 4-PSK.

The parallel processing nature of the genetic algorithm enables the evaluation of many codes simultaneously as the repetition rate is much less than that with the exhaustive search algorithm.

Also, new codes can be found easily, particularly when the constellation and the constraint length are longer, allowing more efficient TCM of large size.

7 Acknowledgment

The authors would like to thank the Departamento de Ingeniería Informática, Santiago University, Chile for their financial support.

8 References

1 ACHA, V., and CARRASCO, R.A.: 'Trellis coded Q^2PSK signals, Part I: AWGN and nonlinear satellite channels', *IEE Proc.-Commun.*, 1994, **141**, (3), pp. 151–158

2 ACHA, V., and CARRASCO, R.A.: 'Trellis coded Q^2PSK signals, Part 2: Land mobile satellite fading channels', *IEE Proc.-Commun.*, 1994, **141**, (3), pp. 159–167

3 BALDINI, R., and FARRELL, P.G.: 'Coded modulation based on ring of integers modulo-q; Part 1: Block codes', *IEE Proc.-Commun.*, 1994, **141**, (3), pp. 129–136

4 BALDINI, R., and FARRELL, P.G.: 'Coded modulation based on ring of integers modulo-q; Part 2: Convolutional codes', *IEE Proc.-Commun.*, 1994, **141**, (3), pp. 137–142

5 BENEDETTO, S.: 'Performance evaluation of trellis-coded modulation schemes', *Proc. IEEE*, 1994, **82**, (6)

6 BIGLIERI, E., and DIVSALAR, D. *et al.*: 'Introduction to trellis-coded modulation with applications' (Macmillan, New York, 1991)

7 BOOKER, L.B. *et al.*: 'Classifier systems and genetic algorithms' (Artificial Intelligence, 1989), **40**, pp. 235–282

8 DELPORT, V.: 'Genetic algorithm for codebook design in vector quantisation', *IEE Electron. Lett.*, 1995, **31**, (2)

9 FORNEY, G.D.: 'Geometrically uniforms codes', *IEEE Trans.*, 1991, **IT–37**, (5), pp. 1241–1260

10 GREFENSTETE, J.J.: 'Optimization of control parameters for genetic algorithms', *IEEE Trans.*, 1986, **SMC–16**, (1), pp. 122–128

11 GOLDEBERG, D.E.: 'Genetic algorithms in search, optimization and machine learning' (Addison–Wesley, 1989)

12 LOPEZ, F.J., CARRASCO, R.A., and FARREL, P.G.: 'Ring-TCM codes for QAM', *Electron. Lett.*, 1992, **28**, (25), pp. 2358–2359

13 ROUANNE, M.: 'An algorithm for computing the distance spectrum of trellis codes', *IEEE J.*, 1989, **SAC–7**, (6), pp. 929–940

14 ZEHAVI, E.: 'On the performance evaluation of trellis code', *IEEE Trans.*, 1987, **IT–33**, (2), pp. 196–202

9 Appendix

9.1 Algorithm 1: Algorithm for select operator

```
select()
{
    Choose randomly α_i in the range 0 ≤ α_i ≤ Σ_{j=1}^{λ} f(a_j^t)
    for {k/∀k ∈ {1, ..., λ},)}
    {
        if((q = max{k.s.t.α_i ≤ Σ_{k=1}^{λ} f(a_k^t)})
        {
            return(q);
        }
    }
}
```

9.2 Algorithm 2: Algorithm for crossover operator

```
crossing()
{
    Choose randomly the coefficient from g_s^(i), ..., g_0^(i),
    f_s, ..., f_1 ∈ S
    if(ρ ≥ ρ_c){
        increment crossover pointer;
        compute crossover pointer;
        }
    else
        crossover pointer = 0; /* no cross, just reproduction */
    gene_counter = 0;
    while(gene_counter < crossover pointer)
    {
        Copy to child1 the genetic information of father;
        Copy to child2 the genetic information of mother;
        increment the gene_counter;
    }
    while(gene_counter < L)
    {
        Copy to child1 the genetic information of mother;
        Copy to child2 the genetic information of father;
        increment the gene_counter;
    }
    share parentage information();
}
```

9.3 Algorithm 3: Algorithm for mutation operator

```
mutate()
{
    k = ring_elements();
    for({k /∀k ∈{1, ..., λ},)})
    {
        if(ρ ≥ ρ_m)
        {
            increment mutation counter;
            alter one bit k = p in the chromosome;
        }
    }
}
```